

Formalizing Brouwer’s Fixed Point Theorem via Scarf’s Lemma in Lean

Anonymous Authors¹

Abstract

We formalize in Lean 4 a combinatorial route from Scarf’s theorem to Brouwer’s fixed point theorem and the existence of mixed Nash equilibria in finite games. The development follows Ivanov’s abstract framework of finite sets equipped with indexed linear orders and colorings, formalizing dominant sets, cells, rooms, doors, and the parity argument for colorful rooms. We then derive Brouwer’s fixed point theorem for the standard simplex through finite grid approximations, compactness, continuity, and vanishing-diameter estimates, and extend the result to finite products of simplices via an explicit embedding–projection construction satisfying a retraction identity. Finally, we apply the product version to mixed strategy spaces of finite games by defining a continuous Nash map whose fixed points are Nash equilibria. The formalization integrates combinatorial, analytic, and game-theoretic reasoning in a unified Lean 4 development.

1. Introduction

Brouwer’s fixed point theorem originated as a fundamental result in topology (Brouwer, 1911) and has since become a central existence tool in analysis, economics, and game theory (Border, 1985). One of its most important applications is the existence of mixed Nash equilibria in finite games (Nash, 2024): the mixed strategy space of a finite game is a finite product of simplices, and a suitably constructed continuous self-map of this space has a fixed point precisely when the corresponding mixed strategy profile is an equilibrium. Classical proofs of Brouwer’s theorem often rely on topological tools such as homology, degree theory, or simplicial approximation. For a formal proof, however, it is useful to expose a route whose finite com-

binatorial core can be separated from the analytic limiting argument.

This paper follows such a route through Scarf’s combinatorial theorem (Scarf, 1982). In Ivanov’s abstract formulation (Ivanov, 2019), Scarf’s theorem concerns a finite set equipped with indexed linear orders and a coloring. The proof is organized around dominant sets, cells, rooms, and doors. The key structural fact is a room–door incidence property: outside doors are incident to exactly one room, while internal doors are incident to exactly two rooms. This incidence structure supports a parity argument, which yields the existence of a colorful room. Applying Scarf’s theorem to increasingly fine grids of the standard simplex gives the finite combinatorial input for Brouwer’s fixed point theorem. The passage from colorful rooms to a fixed point is then obtained by compactness, continuity, and vanishing-diameter estimates. Finally, Brouwer’s theorem for the standard simplex is extended to finite products of simplices by an explicit embedding–projection construction, giving the fixed-point principle needed for the game-theoretic application.

1.1. Lean and Mathlib

Throughout the paper, we use Lean 4 (Moura & Ullrich, 2021) as the proof assistant in which all definitions, theorem statements, and proofs are checked, and Mathlib (The mathlib Community, 2020) as the underlying mathematical library.

Our development builds on Mathlib’s infrastructure for finite types and finite sets, real numbers, continuity, compactness, finite-dimensional spaces, and the standard simplex, while introducing the Scarf-specific objects needed for the formalization, including indexed linear orders, dominant sets, cells, rooms, doors, colorful rooms, and the room–door incidence relation.

Throughout the text, we use the clickable symbol  to link mathematical statements to their corresponding Lean source code.

1.2. Paper Outline

In §2, we describe the formalization of Scarf’s combinatorial theorem: after introducing the indexed-order set-

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

ting and the basic notions of dominant sets, cells, rooms, and doors, we prove the room–door incidence properties and use them to develop the coloring and parity argument for colorful rooms. §3 is devoted to the derivation of Brouwer’s fixed point theorem from Scarf’s theorem: in §3.1, we treat the standard simplex by finite grid approximation, compactness, and continuity, and in §3.2, we extend the result to finite products of simplices through an explicit embedding–projection construction. In §4, we apply the product version of Brouwer’s theorem to the mixed strategy spaces of finite games and derive the existence of mixed Nash equilibria. Finally, we conclude with implementation remarks and future work in §5.

2. Formalizing Scarf’s Combinatorial Theorem

This section contains the finite combinatorial part of the development. We follow Ivanov’s abstract formulation of Scarf’s theorem (Ivanov, 2019), where the input is a finite set equipped with a family of indexed linear orders and a coloring. The proof is organized around three layers. First, dominance turns order-theoretic data into finite cells. Second, rooms and doors provide the incidence structure used for counting. Third, colorings and nearly colorful cells turn this incidence structure into a parity argument producing a colorful room.

2.1. Orders, Dominant Sets, and Cells

We work with a finite type T together with a finite index type I . For each $i \in I$, we are given a linear order $<_i$ on T . The indexed family of orders is the basic object from which dominance is defined. In Lean, this data is bundled as a typeclass [\[1\]](#), so that the notation for comparisons can depend on the index.

Definition 2.1 (Indexed family of linear orders). A family of linear orders on T indexed by I is a collection $(<_i)_{i \in I}$ such that each $<_i$ is a linear order on T .

```
class IndexedLOrder (I T : Type*) where
  IST : I → LinearOrder T
```

Listing 1. Indexed families of linear orders.

Given a nonempty subset $\sigma \subseteq T$ and an index $i \in I$, we write $\min_i \sigma$ for the minimum of σ with respect to $<_i$. Dominance says that every point of T is controlled by at least one of the orders indexed by C . This is the order-theoretic condition that later forces the relevant finite sets to have the correct cardinalities.

Definition 2.2 (Dominant set). Let C be a nonempty subset of I . A subset σ of T is dominant with respect to C if, for every element y of T , one can find an index i in C such that y is below every element of σ in the order $<_i$.

When σ is nonempty, this is equivalent to saying that no element of T is strictly larger than all the minima $\min_i \sigma$ for i in C .

```
def isDominant := ∀ y, ∃ i ∈ C, ∀ x ∈ σ,
  y ≤ [i] x
```

Listing 2. Dominance.

We adopt the convention that $\emptyset \subset T$ is dominant with respect to every non-empty subset $C \subset I$ [\[2\]](#). The following lemma collects the basic closure and cardinality facts about dominant sets. These facts are used repeatedly when passing between cells, rooms, and doors.

Lemma 2.3 (Basic properties of dominant sets). If σ is nonempty and dominant with respect to C , then $\sigma = \{\min_i \sigma \mid i \in C\}$, and hence $|\sigma| \leq |C|$ [\[3\]](#) and hence $|\sigma| \leq |C|$ [\[4\]](#). If $\tau \subset \sigma$, then τ is dominant with respect to C [\[5\]](#), and if $C \subset D$, then σ is dominant with respect to D [\[6\]](#).

The preceding lemma allows us to treat dominant pairs (σ, C) as cells of a finite combinatorial object. Rooms are the full-dimensional cells, while doors are their codimension-one boundaries in the counting argument.

Definition 2.4 (Cells, rooms, and doors). A *cell* is a pair (σ, C) where σ is dominant with respect to C . A cell is called a *room* if $|C| = |\sigma|$, and a *door* if $|C| = |\sigma| + 1$.

```
abbrev isCell := isDominant σ C

abbrev isRoom := isCell σ C ∧ C.card = σ.card

abbrev isDoor := isCell σ C ∧ C.card
               = σ.card + 1
```

Listing 3. Cells, rooms, and doors.

2.2. Rooms, Doors, and the Two-Room Property

The next step is to make precise how rooms meet along doors. In the formalization, this is captured by the predicate `isDoorof` [\[7\]](#), which records that a door is obtained from a room either by removing an element from the underlying set or by adding a color. This relation is the local incidence relation used later in the parity count.

Definition 2.5 (Outside and internal doors). A door (τ, D) is called an *outside door* if $\tau = \emptyset$, and an *internal door* otherwise.

```
variable (τ D) in
abbrev isOutsideDoor := IST.isDoor τ D ∧ τ
                      = Finset.empty

variable (τ D) in
abbrev isInternalDoor := IST.isDoor τ D
```

$\wedge \tau.\text{Nonempty}$

Listing 4. outside door and internal door.

The following two lemmas give the degree structure of this incidence relation. Outside doors behave as boundary objects: they have degree one. Internal doors behave as shared faces: they have degree two.

Lemma 2.6 (Outside-door property). *Suppose that $|D| = 1$. Then $(\emptyset, D)$ is an outside door and is a door of exactly one room. Every outside door has this form.*

Lemma 2.7 (Two-room property). *Every internal door belongs to exactly two rooms.*

Thus the rooms and doors form a finite graph-like incidence structure: starting from an outside door, one enters a unique room, and every time one crosses an internal door there is a unique other room on the opposite side. This is the combinatorial shape behind the later path-following and parity argument.

The proof of the two-room property is the most involved local part of the Scarf formalization. It analyzes an internal door (τ, D) via two complementary constructions: either removing a color from D , or extending τ by a suitable element. These are implemented in Lean via the auxiliary constructions `M_set` and maximal elements `m_element`, which ensure that both constructions yield valid rooms. In the paper, we use the two-room property as a black-box incidence theorem; in Lean, the proof supplies the exact finite-set witnesses needed to make the incidence count well typed.

2.3. Colorings and the Parity Argument

We now add the coloring data. The coloring selects, for each element of T , an index in I . A colorful cell is one whose index set is exactly the image of its underlying set under this coloring. Nearly colorful cells differ from being colorful by exactly one missing color; the missing color is recorded as the type. This typed version is useful because the parity argument counts objects of a fixed type.

Definition 2.8 (Coloring). Fix a coloring $c : T \rightarrow I$. A cell (σ, C) is called *colorful* if $C = c(\sigma)$, and *nearly colorful* if $|C \setminus c(\sigma)| = 1$. For a nearly colorful cell (σ, C) , the unique element of $C \setminus c(\sigma)$ is called its *type*.

```
def isColorful : Prop := IST.isCell σ C
  ∧ σ.image c = C

def isNearlyColorful : Prop := IST.isCell σ C
  ∧ (C \ σ.image c).card = 1

def isTypedNC (i : I) (σ : Finset T)
  (C : Finset I) : Prop := IST.isCell σ C
  ∧ (C \ (σ.image c)) = {i}
```

Listing 5. Colorful and nearly colorful cells.

The next lemmas connect the uncolored incidence structure with the coloring data. They identify the boundary terms, describe how colorful rooms contribute nearly colorful doors, and show that nearly colorful rooms have the expected number of nearly colorful doors.

Lemma 2.9 (Outside doors and doors of colorful rooms). *For every $i \in I$ there is exactly one outside door of the type i . Every door of a colorful room is nearly colorful and there is exactly one door of each type among them.*

Lemma 2.10 (Nearly colorful door transition). *Suppose that (τ, D) is a nearly colorful door of a room (σ, C) . Then (σ, C) is either colorful, or nearly colorful of the same type as (τ, D) .*

Lemma 2.11 (Nearly colorful cardinality properties). *If (σ, C) is a nearly colorful room, then $|c(\sigma)|$ is equal either to $|\sigma|$ or to $|\sigma| - 1$. If (σ, C) is a nearly colorful door, then $c(\sigma) \subset C$.*

Lemma 2.12 (Doors of nearly colorful rooms). *If (σ, C) is a nearly colorful room, then (σ, C) has two nearly colorful doors.*

Together, these lemmas turn the room–door incidence relation into a parity count. Fixing a type i , the unique outside door of type i contributes one boundary term. Internal nearly colorful doors are counted twice because of the two-room property. Nearly colorful rooms also contribute an even number of typed doors. Therefore the remaining contribution comes from colorful rooms, forcing the relevant colorful-room count to be odd.

Theorem 2.13 (Scarf’s combinatorial theorem). *For every coloring $c : T \rightarrow I$, there exists a colorful room. Moreover, the number of colorful rooms is odd.*

This theorem completes the finite combinatorial core of the formalization. In the next section, the finite set T will be instantiated by a grid in the standard simplex. The colorful rooms supplied by Scarf’s theorem then become the discrete approximations from which Brouwer fixed points are obtained by a limiting argument.

3. From Scarf’s Theorem to Brouwer’s Fixed Point Theorem

This section derives Brouwer’s fixed point theorem from the formalized Scarf theorem. The argument has two stages. We first prove Brouwer’s theorem for the standard simplex by applying Scarf’s theorem to increasingly fine finite grids and then passing to a limit. We then obtain the corresponding theorem for finite products of simplices

by reducing the product case to the standard-simplex case through an explicit embedding–projection construction.

3.1. Brouwer’s Theorem on the Standard Simplex

Fix a positive integer n , and let Δ^{n-1} be the standard simplex with coordinates indexed by $\text{Fin } n$. The goal is to prove that every continuous self-map

$$f : \Delta^{n-1} \rightarrow \Delta^{n-1}$$

has a fixed point. The proof does not use Scarf’s theorem directly on the simplex itself. Instead, for each positive integer l , we work with a finite grid T_l of rational points whose coordinates have denominator l . Informally,

$$T_l = \left\{ x \in \Delta^{n-1} \mid x_i \in \frac{1}{l}\mathbb{Z} \text{ for all } i \right\}.$$

In Lean, the grid is represented by integer coordinates in $\text{Fin } (l+1)$ whose sum is l ; the corresponding point of the simplex is obtained by dividing each coordinate by l . The subtype TT defines the finite grid, while TTtoStdSimplex and the coercion instance TT.CoStdSimplex realize a grid point as an element of the standard simplex.

```
abbrev TT := {x : Πl ( _ : Fin n), Fin (l+1) | ∑  
  i, (x i : ℕ) = l}  
  
def TTtoStdSimplex (x : TT n l) : stdSimplex ℝ  
  (Fin n)
```

Listing 6. The discrete grid used to approximate the simplex.

3.1.1. THE ORDERED GRID AND DOMINANCE ESTIMATES

To apply Scarf’s theorem, each grid T_l is equipped with an indexed family of linear orders, one order for each coordinate. For a fixed coordinate i , the order first compares the i -th coordinate and then uses a lexicographic tie-breaker on the whole grid point. This is encoded by TT.ILt and assembled into the indexed linear order TT.ILO . These orders are chosen so that dominance in the sense of Scarf’s theorem imposes strong metric restrictions on the corresponding grid points. This is the point at which the abstract combinatorial theorem becomes a quantitative statement about the simplex.

```
abbrev TT.ILt (x y : TT n l) :=  
  toLex (x i, x) < toLex (y i, y)  
  
instance TT.ILO : IndexedLOrder (Fin n) (TT n l)
```

Listing 7. The indexed linear orders on the grid.

The key quantitative input is that dominance on the grid gives metric control. A dominant set has small diameter, and the coordinates outside its color set are uniformly

small. These estimates are the bridge between the finite combinatorial theorem and the limiting argument.

Theorem 3.1 (Dominance estimates on the grid). *Let $\sigma \subseteq T_l$ and $C \subseteq \text{Fin } n$. If σ is dominant with respect to C , then*

$$|x_i - y_i| < \frac{2(n+1)}{l}$$

for every $x, y \in \sigma$ and every $i \in \text{Fin } n$. Moreover, if $i \notin C$, then

$$x_i < \frac{n+1}{l}$$

for every $x \in \sigma$.

Thus, as l tends to infinity, dominant rooms become small, and the coordinates not belonging to their color sets vanish in the limit.

3.1.2. THE COLORING INDUCED BY A SELF-MAP

Let $f : \Delta^{n-1} \rightarrow \Delta^{n-1}$ be continuous. For any point $x \in \Delta^{n-1}$, the coordinate sums of x and $f(x)$ are both equal to 1. Hence not all coordinates can strictly decrease: there exists an index i such that

$$x_i \leq f(x)_i.$$

The formalization packages this elementary observation as the nonemptiness lemma `stdSimplex.upidx` and the choice function `stdSimplex.pick` on the simplex. Restricting this choice to the grid T_l gives the coloring required by Scarf’s theorem, namely `Fcolor`.

```
noncomputable def stdSimplex.pick  
  (x y : stdSimplex ℝ (Fin n))  
  
def Fcolor (x : TT n l) : Fin n :=  
  stdSimplex.pick x (f x)
```

Listing 8. The coloring induced by a continuous self-map.

Applying the formalized Scarf theorem to `Fcolor` produces a colorful room on each grid T_l . In Lean, these rooms are chosen as a sequence `room_seq`, and a representative point from each room is chosen as `room_point_seq`. This sequence of rooms is the bridge between the finite combinatorial theorem and the limiting proof of Brouwer’s theorem.

```
def room_seq (l' : ℕ) :=  
  let l : PNat := ⟨l'+1, Nat.zero_lt_succ _⟩  
  Classical.choice (TT.ILO.Scarf (@Fcolor n l  
    f)).to_subtype  
  
def room_point_seq (l' : ℕ) :=  
  pick_colorful_point  
  (Finset.mem_filter.1 (room_seq f l')).2.2 |>.1
```

Listing 9. Choosing colorful rooms and representative points.

3.1.3. COMPACTNESS AND THE LIMITING ARGUMENT

Choose one representative point from the colorful room obtained at each grid level. Since there are only finitely many possible color sets, we first pass to a subsequence on which the color set of the chosen colorful rooms is constant. This finite-image subsequence extraction is formalized by `exists_subseq_constant_of_finite_image`; the resulting constant color set and order embedding are packaged as `gpkg` and `g1`. We then use compactness of the standard simplex to extract a convergent subsequence of the corresponding room points; this step is packaged in Lean as `hpkg_aux` and `hpkg`. Let $z \in \Delta^{n-1}$ denote the resulting limit, and write the constant color set as C .

The dominance estimates now control the limit. First, the diameters of the selected colorful rooms tend to zero. Therefore every point in the same selected room converges to the same limit z . Second, for each coordinate $i \notin C$, the outside-coordinate estimate gives convergence to zero, and hence

$$z_i = 0 \quad \text{for all } i \notin C.$$

Since z lies in the simplex, it follows that

$$\sum_{i \in C} z_i = 1.$$

On the other hand, the coloring rule says that for each color $i \in C$, the chosen room contains grid points whose i -th coordinate does not decrease under f . Passing to the limit and using continuity of f gives

$$f(z)_i \geq z_i \quad \text{for all } i \in C.$$

Finally, $f(z)$ is also a point of the simplex. The inequalities on C , together with $\sum_{i \in C} z_i = 1$, force equality on all coordinates in C and force all coordinates outside C to be zero. Hence $f(z) = z$.

Theorem 3.2 (Brouwer's fixed point theorem for the standard simplex). *Every continuous map $f : \Delta^{n-1} \rightarrow \Delta^{n-1}$ has a fixed point.*

3.2. Brouwer's Theorem on Products of Simplices

We next extend Brouwer's fixed point theorem from the standard simplex to finite products of simplices. This is the form needed for the game-theoretic application, where a mixed strategy profile is a point in a product of simplices. The formal proof reduces the product case to the standard-simplex case by constructing an explicit embedding–projection pair between the product and a larger simplex.

3.2.1. REDUCING PRODUCTS TO A STANDARD SIMPLEX

Let I be a finite index type, and let $\text{card}(i)$ be a positive integer for each $i \in I$. The product space considered in

Lean is

$$\prod_{i \in I} \Delta^{\text{card}(i)-1}.$$

It is represented by the type `ProductSimplices card`. The ambient simplex has total coordinate number

$$N = \sum_{i \in I} \text{card}(i),$$

and is represented by `BigSimplex card`.

The main bookkeeping issue is that coordinates in the ambient simplex are indexed by a single flat index, while coordinates in the product are indexed by a pair consisting of a component $i \in I$ and a coordinate inside the i -th simplex. In Lean, this conversion is handled by `index_split` and `index_combine`. These maps allow the formalization to move between the block decomposition of the product and the flat coordinate system of the larger simplex.

3.2.2. THE EMBEDDING-PROJECTION CONSTRUCTION

The embedding sends a point of the product of simplices to a point of the ambient simplex by placing each component simplex into its corresponding coordinate block. The i -th block is rescaled by the weight $\text{card}(i)/N$, so that the total sum of all coordinates is one. This construction is formalized as `embed_from_product`.

The projection in the opposite direction is more delicate. Starting from a point of the ambient simplex, the natural idea is to recover each component simplex by normalizing the corresponding coordinate block. However, this normalization is not well-defined if one of the blocks has total mass zero. To avoid this degenerate case, the formal construction first pushes the ambient point toward a fixed uniform point `z_uniform`. This auxiliary operation is implemented by `pushTowardsZ`, together with the definitions `deficit` and `tPush`. The key property is that after this push, every coordinate block has positive total mass. The projection is then obtained by normalizing each block, and is formalized as `project_to_product`.

The central property of the construction is the retraction identity

$$\text{project_to_product} \circ \text{embed_from_product} = \text{id}.$$

For points already coming from the product, each coordinate block has the prescribed positive total mass, so the projection recovers the original product point. This identity is formalized by `project_embed_id`.

3.2.3. THE PRODUCT FIXED-POINT THEOREM

It remains to transfer Brouwer's theorem from the ambient simplex back to the product. The embedding and projection are proved continuous as `embed_continuous` and

project_continuous [↗](#). Given a continuous map

$$f : \prod_{i \in I} \Delta^{\text{card}(i)-1} \rightarrow \prod_{i \in I} \Delta^{\text{card}(i)-1},$$

we form a continuous self-map of the ambient simplex by composing projection, f , and embedding:

$$F = \text{embed_from_product} \circ f \circ \text{project_to_product}.$$

The standard-simplex version of Brouwer’s theorem gives a fixed point of F . Projecting this fixed point back to the product and using the retraction identity gives a fixed point of f .

Theorem 3.3 (Brouwer fixed point theorem for products of simplices). *Every continuous self-map of a finite product of simplices has a fixed point* [↗](#).

4. Nash Equilibrium Existence in Finite Games

We now apply the product version of Brouwer’s fixed point theorem to finite strategic-form games. The main point is that, after passing from pure strategies to mixed strategies, the strategy space becomes a finite product of simplices. We then construct the standard Nash map on this product, prove that it is continuous, and show that every fixed point of this map satisfies the mixed Nash equilibrium condition.

4.1. Finite Games and Mixed Strategy Profiles

A strategic-form game consists of a nonempty type of players, a nonempty type of pure strategies for each player, and a payoff function for each player. In Lean, the basic structure is kept deliberately general: finiteness is added only in the derived structure `FinGame`. This separation allows the same notion of game to be reused before and after passing to mixed strategies.

```
structure Game where
  I : Type*
  HI : Inhabited I
  SS : I → Type*
  HSS (i : I) : Inhabited (SS i)
  g : I → (Π i, SS i) → ℝ
attribute [instance] Game.HI Game.HSS
```

Listing 10. Strategic-form games

For a game $G = (I, (S^i)_{i \in I}, (g^i)_{i \in I})$, a pure Nash equilibrium is a strategy profile $s = (s^i)_{i \in I}$ such that no player can improve their payoff by changing only their own strategy: for every player i and every alternative pure strategy $t^i \in S^i$,

$$g^i(t^i, s^{-i}) \leq g^i(s^i, s^{-i}).$$

This condition is formalized as the basic equilibrium predicate for games. [↗](#)

For the fixed-point argument we restrict to finite games. A finite game has finitely many players and finitely many pure strategies for each player. Its mixed strategy space is the product of standard simplices

$$\Delta(G) = \prod_{i \in I} \Delta(S^i),$$

where a point $\sigma \in \Delta(G)$ assigns to every player i a probability distribution σ^i over S^i . In Lean this is represented by the dependent function type `mixedS`.

```
structure FinGame extends Game where
  FinI : Fintype I
  FinSS : ∀ i : I, Fintype (SS i)
attribute [instance] FinGame.FinI FinGame.FinSS
variable {G : FinGame}
instance {G : FinGame} : Fintype G.I := G.FinI
instance {G : FinGame} {i : G.I} : Fintype (G.SS i) := G.FinSS i
variable (G) in
abbrev mixedS := (i : G.I) → stdSimplex ℝ (G.SS i)
```

Listing 11. Finite games and mixed strategy profiles

The pure payoff function extends to mixed profiles by taking expected payoff. For a player i , the mixed payoff `mixed_g i σ` is the multilinear extension of the pure payoff function, obtained by summing over pure strategy profiles with weights given by the players’ mixed strategies. [↗](#) The formalization also proves the algebraic lemmas needed to treat this extension as linear in the relevant coordinates. [↗](#)

It is convenient to package the mixed extension itself as another game, whose strategy type for player i is the simplex over S^i and whose payoff function is the expected payoff.

```
def FinGame2MixedGame (G : FinGame) : Game := {
  I := G.I
  HI := G.HI
  SS := fun i => S (G.SS i)
  HSS := inferInstance
  g := mixed_g
}
```

Listing 12. The mixed extension of a finite game

A mixed Nash equilibrium is then a Nash equilibrium of this mixed game: a mixed profile σ^* such that no player can obtain a larger expected payoff by replacing σ^{*i} with another mixed strategy while all other players keep their components fixed. [↗](#)

4.2. The Nash Map

Following the standard Brouwer proof of Nash’s theorem, we define a continuous self-map on the mixed strategy space. For a mixed profile σ , player i , and pure strategy $s^i \in S^i$, let

$$A_i(s^i, \sigma) = (g^i(s^i, \sigma^{-i}) - g^i(\sigma))^+, \quad a^+ = \max(a, 0).$$

The Nash map $F : \Delta(G) \rightarrow \Delta(G)$ is defined coordinate-wise by

$$F(\sigma)^i(s^i) = \frac{\sigma^i(s^i) + A_i(s^i, \sigma)}{1 + \sum_{t^i \in S^i} A_i(t^i, \sigma)}.$$

The denominator is positive, and the coordinates in each player block sum to one; hence $F(\sigma)^i$ is again a mixed strategy. The Lean definition mirrors this formula by first defining the unnormalized numerator and then normalizing it inside the simplex certificate.

```

noncomputable abbrev g_function (i : G.I) (σ :
  G.mixedS) (a : G.SS i) : ℝ :=
  σ i a + max 0 (mixed_g i (Function.update σ i
    (stdSimplex.pure a)) - mixed_g i σ)

noncomputable abbrev nash_map_aux (σ :
  G.mixedS) (i : G.I) (a : G.SS i) : ℝ :=
  g_function i σ a / ∑ b : G.SS i, g_function
    i σ b

noncomputable def nash_map (σ : G.mixedS) :
  G.mixedS :=
  fun i ↦ ⟨nash_map_aux σ i, nash_map_cert σ i⟩
    
```

Listing 13. The Nash map

The proof has two parts. First, we prove that `nash_map` is continuous. Mathematically this follows from the finiteness of the strategy sets and the fact that the formula is built from finite sums, subtraction, max, and division by a positive denominator. In Lean, the main work is to make the continuity of the expected payoff function explicit before assembling the continuity proof for the whole map. [\[1\]](#)

Second, applying the product version of Brouwer’s theorem to `nash_map` gives a fixed point σ . [\[2\]](#) It remains to show that this fixed point is a mixed Nash equilibrium. If all positive payoff improvements vanish, then every pure deviation has payoff at most $g^i(\sigma)$, and therefore no mixed deviation can improve the payoff. If some positive improvement were available at a fixed point, the normalization equation defining the Nash map would force a contradiction with the expected payoff identity. Thus every fixed point of the Nash map satisfies the mixed Nash equilibrium condition.

Theorem 4.1 (Existence of mixed Nash equilibria). *Every finite strategic-form game has a mixed Nash equilibrium.* [\[3\]](#)

5. Discussion

5.1. Implementation Remarks

The formalization is organized as a chain of reusable components, moving from finite combinatorics to fixed-point theory and then to equilibrium existence. The Scarf part is the most combinatorial. Although the informal proof is guided by an intuitive room–door picture, the Lean development requires explicit definitions of indexed orders, dominant sets, cells, rooms, doors, and the incidence relation between them. Many steps that are implicit on paper become finite-set and cardinality arguments in Lean. In particular, the two-room property requires a careful case analysis of outside and internal doors, together with explicit witnesses for the possible rooms adjacent to a given door.

The Brouwer part illustrates a different kind of formalization difficulty. The proof must connect the finite output of Scarf’s theorem with an analytic limiting argument. This requires defining finite grid approximations of the simplex, constructing the coloring induced by a continuous self-map, proving quantitative diameter estimates, extracting convergent subsequences, and transporting coordinate inequalities to the limit. The product-of-simplices version adds further bookkeeping: points in a product of simplices must be related to points in a larger ambient simplex, and the projection back to the product must avoid degenerate zero-block cases. Our embedding–projection construction makes this reduction explicit and proves the required retraction identity inside Lean.

The Nash part then uses the product version of Brouwer’s theorem as a reusable fixed-point principle. Once mixed strategy profiles are represented as products of simplices, the remaining tasks are to define expected payoffs, construct the Nash map, prove its continuity, and show that its fixed points satisfy the mixed Nash equilibrium condition. This separation is useful: the game-theoretic proof can be developed mostly in terms of payoff functions and best-response inequalities, while the topological fixed-point argument is isolated in the preceding Brouwer formalization.

5.2. Future Work

A natural direction for future work is to use this development as the starting point for a broader Lean library of game theory. The present formalization establishes several foundational components: finite games, mixed strategies as products of simplices, expected payoffs, Nash equilibria, and the fixed-point argument needed for equilibrium existence. These components could serve as a formal backbone for later developments covering zero-sum games, extensive-form games, repeated games, equilibrium refinements, and mechanism-design-style constructions.

Our longer-term goal is to formalize the material of a full

textbook on game theory. A textbook-scale project would require not only more theorems, but also a more systematic hierarchy of reusable definitions, so that later chapters can build on earlier formalized notions without repeatedly rebuilding the same infrastructure. The Scarf–Brouwer–Nash pipeline provides a useful test case because it combines finite combinatorics, topology, analysis, and game theory in a single proof development.

Auto-formalization is an important tool for making such a project feasible. Recent work on neural theorem proving and formal-mathematics benchmarks has shown that language models can contribute to proof search, statement translation, and proof repair, while still relying on proof assistants as the final checkers (Polu & Sutskever, 2020; Zheng et al., 2021). In our setting, a possible workflow is to combine human-written mathematical outlines with model-generated Lean drafts: the system would translate textbook definitions and proof sketches into Lean, retrieve relevant Mathlib lemmas and previously formalized game-theoretic results, generate auxiliary lemmas when direct proof attempts fail, and iteratively repair proof scripts using Lean error messages.

In addition to expanding the library itself, future work will involve building benchmarks from the formalization process. Each definition, theorem statement, and proof step in a game theory text can be treated as an evaluation unit for statement translation, lemma retrieval, proof synthesis, and proof repair. In this way, the project can contribute both to the formalization of game theory and to the development of practical auto-formalization methods for large mathematical texts.

Impact Statement

This paper contributes to formalized mathematics and machine-checkable reasoning in proof assistants. Its results are primarily foundational and theoretical, focusing on the formal verification of a route from Scarf’s combinatorial theorem to Brouwer’s fixed point theorem and Nash equilibrium existence. We do not anticipate direct negative societal impacts from the results themselves. More broadly, the work may support the development of reliable mathematical libraries and auto-formalization tools; as with other formal reasoning technologies, their downstream use should be evaluated in the context of the systems in which they are deployed.

References

Border, K. C. *Fixed point theorems with applications to economics and game theory*. Cambridge university press, 1985.

Brouwer, L. E. Beweis der invarianz der dimensionenzahl. *Mathematische Annalen*, 70(2):161–165, 1911.

Ivanov, N. V. Beyond Sperner’s lemma. *arXiv preprint arXiv:1902.00827*, 2019.

Moura, L. d. and Ullrich, S. The lean 4 theorem prover and programming language. In *International Conference on Automated Deduction*, pp. 625–635. Springer, 2021.

Nash, J. F. Non-cooperative games. In *The Foundations of Price Theory Vol 4*, pp. 329–340. Routledge, 2024.

Polu, S. and Sutskever, I. Generative language modeling for automated theorem proving. *arXiv preprint arXiv:2009.03393*, 2020.

Scarf, H. E. The computation of equilibrium prices: an exposition. *Handbook of mathematical economics*, 2: 1007–1061, 1982.

The mathlib Community. The Lean Mathematical Library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs, CPP 2020, New Orleans, LA, USA, January 2020*. ACM. doi: 10.1145/3372885.3373824. URL <https://doi.org/10.1145/3372885.3373824>.

Zheng, K., Han, J. M., and Polu, S. Minif2f: a cross-system benchmark for formal olympiad-level mathematics. *arXiv preprint arXiv:2109.00110*, 2021.